

**LAPORAN STUDI KASUS**  
**E-COMMERCE ORDER RECOMENDATION ENGINE**

Tugas ini disusun untuk memenuhi salah satu persyaratan memperoleh nilai Mata kuliah  
Algoritma dan Struktur Data

Dosen Pengampu Mata Kuliah : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT..



Disusun oleh :

|                           |               |
|---------------------------|---------------|
| Muhammad Raykhan Fadillah | (25051030057) |
| Dewangga Da'i Prabawa     | (25051030058) |
| Dandy Faisal Bimasena     | (25051030062) |
| Muhammad Fathir Al Malik  | (25051030067) |

PROGRAM STUDI TEKNIK ELEKTRO  
DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO  
FAKULTAS TEKNIK  
UNIVERSITAS NEGERI YOGYAKARTA

2026

## DAFTAR ISI

|                                                                                                                                                                                              |   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| LAPORAN STUDI KASUS.....                                                                                                                                                                     | 1 |
| DAFTAR ISI.....                                                                                                                                                                              | 2 |
| DAFTAR GAMBAR .....                                                                                                                                                                          | 4 |
| BAB I PENDAHULUAN.....                                                                                                                                                                       | 5 |
| 1.1    Latar belakang.....                                                                                                                                                                   | 5 |
| 1.2    Tujuan Sistem.....                                                                                                                                                                    | 5 |
| a.    Membuat sistem e-commerce yang mampu mengelola pesanan pelanggan secara cepat, efisien, dan terstruktur menggunakan konsep struktur data. ....                                         | 5 |
| b.    Mengimplementasikan sistem antrian berbasis prioritas untuk membedakan pelayanan pelanggan PREMIUM, REGULAR, dan ECONOMY agar proses order berjalan lebih optimal. ....                | 5 |
| c.    Mengimplementasikan struktur data seperti Queue, Stack, Binary Search Tree (BST), dan Graph untuk pengelolaan order, katalog produk, riwayat transaksi, serta rekomendasi produk. .... | 5 |
| d.    Menganalisis kompleksitas algoritma (Big-O) pada setiap operasi utama untuk mengetahui efisiensi dan performa sistem. ....                                                             | 5 |
| 1.3    Batasan Implementasi .....                                                                                                                                                            | 5 |
| 1.4    Diagram Arsitektur Sistem.....                                                                                                                                                        | 5 |
| BAB II PERANCANGAN SISTEM.....                                                                                                                                                               | 7 |
| 2.1    Gambaran Umum Sistem.....                                                                                                                                                             | 7 |
| 2.2    Pemilihan Struktur Data dan Justifikasi.....                                                                                                                                          | 7 |
| a.    Queue .....                                                                                                                                                                            | 7 |
| b.    Stack.....                                                                                                                                                                             | 7 |
| c.    Binary Search Tree (BST).....                                                                                                                                                          | 7 |
| d.    Graph.....                                                                                                                                                                             | 8 |
| 2.3    Diagram Interaksi Antar Modul .....                                                                                                                                                   | 8 |

|                                          |                                             |    |
|------------------------------------------|---------------------------------------------|----|
| 2.4                                      | Algoritma Utama Sistem.....                 | 8  |
| 2.5                                      | Analisis Kompleksitas .....                 | 9  |
| BAB III IMPLEMENTASI.....                |                                             | 10 |
| 3.1                                      | Penjelasan Code Permodul.....               | 10 |
| a.                                       | Modul 1 : Multi-Priority Order Queue.....   | 10 |
| b.                                       | Modul 2 : BST Katalog Produk .....          | 10 |
| c.                                       | Modul 3 : Graph Rekomendasi .....           | 10 |
| d.                                       | Modul 4 : Stack Riwayat Transaksi .....     | 11 |
| e.                                       | Modul 5 : Sorting Laporan Harian .....      | 11 |
| f.                                       | Modul 6 : CLI E-Commerce .....              | 11 |
| 3.2                                      | Screenshot CLI berjalan.....                | 12 |
| 3.3                                      | Contoh input / output nya .....             | 12 |
| BAB IV ANALISIS BIG-O .....              |                                             | 13 |
| 4.1                                      | Analisis Kompleksitas Waktu dan Ruang ..... | 13 |
| 4.2                                      | Tabel Ringkasan Kompleksitas .....          | 14 |
| 4.3                                      | Perbandingan Teoritis dan Eksperimen.....   | 15 |
| BAB V HASIL EKSPERIMEN.....              |                                             | 16 |
| 5.1                                      | Tabel Runtime Untuk 3+ Ukuran Data .....    | 16 |
| 5.2                                      | Grafik tren .....                           | 16 |
| 5.3                                      | Diskusi .....                               | 17 |
| BAB VI JAWABAN PERTANYAAN ANALISIS ..... |                                             | 18 |
| 6.1                                      | Jawaban Pertanyaan Analisis .....           | 18 |
| BAB VII KESIMPULAN .....                 |                                             | 22 |
| 7.1                                      | Ringkasan Pencapaian.....                   | 22 |
| 7.2                                      | Refleksi Trade-off Desain .....             | 22 |
| 7.3                                      | Saran Pengembangan .....                    | 23 |
| LAMPIRAN.....                            |                                             | 24 |

## DAFTAR GAMBAR

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 1. 1 Flowchart arsitekrur sistem .....   | 3  |
| Gambar 1. 2 Flowchart arsitekrur sistem 2 ..... | 3  |
| Gambar 2. 1 Interaksi Antar Modul .....         | 6  |
| Gambar 3. 1 CLI berjalan .....                  | 10 |
| Gambar 5. 1 Grafik tren .....                   | 14 |

# BAB I

## PENDAHULUAN

### 1.1 Latar belakang

Perkembangan teknologi informasi dan internet pada era modern saat ini mengalami kemajuan yang sangat pesat, terutama dalam bidang perdagangan elektronik (*e-commerce*). Kehadiran berbagai platform belanja online telah mengubah cara masyarakat melakukan transaksi jual beli menjadi lebih praktis, cepat, dan mudah diakses kapan saja. Menurut teori struktur data, data perlu diatur dan disimpan secara sistematis agar dapat diproses dengan efisien. Struktur data merupakan cara untuk menyusun dan menyimpan data dalam komputer sehingga memudahkan proses akses dan pengolahan informasi. Dengan semakin meningkatnya jumlah pengguna *e-commerce*, jumlah transaksi dan data pesanan juga terus bertambah setiap harinya. Kondisi ini menjadi lebih kompleks ketika terjadi *flash sale*, diskon besar-besaran, promo hari raya, atau event tertentu yang menyebabkan lonjakan pesanan dalam waktu singkat. Jika sistem tidak mampu mengelola data dengan baik, maka dapat terjadi penumpukan order, keterlambatan pelayanan, dan menurunnya performa sistem.

Selain pengelolaan pesanan, sistem rekomendasi produk juga menjadi tantangan penting dalam perkembangan *e-commerce* modern. Banyaknya produk yang tersedia sering membuat pelanggan kesulitan menemukan barang yang sesuai dengan kebutuhan dan minat mereka. Oleh karena itu, dibutuhkan sistem rekomendasi yang mampu membantu pengguna menemukan produk dengan lebih cepat dan tepat. Di sisi lain, terdapat pengelompokan pelanggan seperti pelanggan premium dan pelanggan reguler yang membutuhkan tingkat pelayanan berbeda. Pelanggan premium biasanya diprioritaskan agar memperoleh pelayanan yang lebih cepat dibanding pelanggan biasa. Menurut Charisma Tubagus Setyobudhi (2023) dalam buku *Pemrograman Struktur Data*, pemilihan struktur data yang tepat dapat membuat program berjalan lebih efektif, cepat, dan efisien dalam pengelolaan data. Hal ini menunjukkan bahwa penggunaan struktur data sangat berpengaruh terhadap performa sistem, terutama dalam aplikasi yang menangani data besar seperti *e-commerce*.

Berdasarkan permasalahan tersebut, dibutuhkan sebuah sistem yang mampu mengatur antrian pesanan, menentukan prioritas pelanggan, serta membantu proses rekomendasi produk secara efisien. Struktur data seperti Queue, Priority Queue, Linked List, dan Stack dapat digunakan untuk mengelola data order dan prioritas layanan dengan lebih

terorganisir. Queue dapat membantu pengelolaan antrian pesanan berdasarkan urutan, sedangkan Priority Queue dapat digunakan untuk memprioritaskan pelanggan premium. Dengan penerapan struktur data yang tepat, sistem dapat bekerja lebih cepat, stabil, dan mampu menangani jumlah data yang besar. Oleh karena itu, dibuatlah proyek “**E-Commerce Order Recommendation Engine**” sebagai sistem yang dirancang untuk membantu pengelolaan pesanan dan rekomendasi produk secara efektif dengan memanfaatkan konsep struktur data sehingga proses transaksi pada platform e-commerce dapat berjalan lebih lancar, efisien, dan terstruktur dengan baik.

## 1.2 Tujuan Sistem

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah pada proyek “Commerce Order Management Recommendation” adalah sebagai berikut:

- a. Membuat sistem e-commerce yang mampu mengelola pesanan pelanggan secara cepat, efisien, dan terstruktur menggunakan konsep struktur data.
- b. Mengimplementasikan sistem antrian berbasis prioritas untuk membedakan pelayanan pelanggan PREMIUM, REGULAR, dan ECONOMY agar proses order berjalan lebih optimal.
- c. Mengimplementasikan struktur data seperti Queue, Stack, Binary Search Tree (BST), dan Graph untuk pengelolaan order, katalog produk, riwayat transaksi, serta rekomendasi produk.
- d. Menganalisis kompleksitas algoritma (Big-O) pada setiap operasi utama untuk mengetahui efisiensi dan performa sistem.

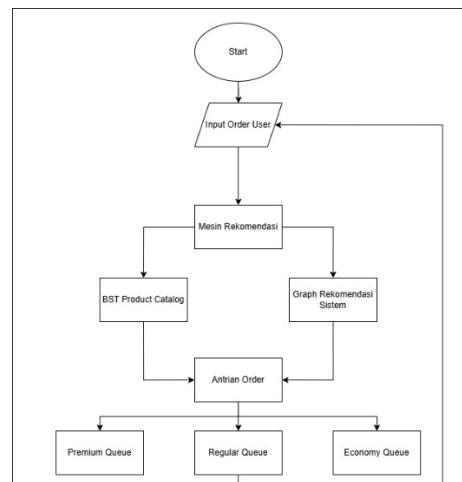
## 1.3 Batasan Implementasi

Tujuan dari proyek “**Commerce Order Management Recommendation**” adalah sebagai berikut:

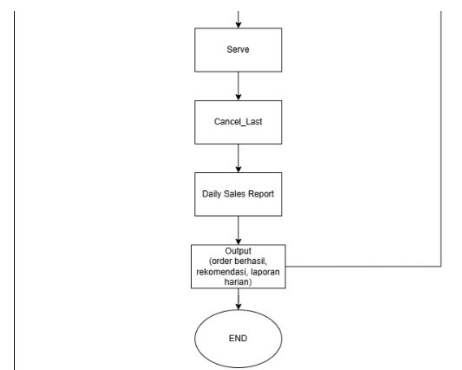
- a. Sistem hanya dijalankan menggunakan antarmuka berbasis CLI (*Command Line Interface*) atau terminal tanpa menggunakan tampilan GUI.
- b. Sistem hanya mendukung pengelompokan pelanggan ke dalam tiga tier, yaitu PREMIUM, REGULAR, dan ECONOMY untuk pengaturan prioritas pelayanan order.
- c. Sistem tidak menggunakan database sehingga seluruh data produk, order, dan riwayat transaksi hanya disimpan sementara selama program berjalan.
- d. Sistem hanya mengimplementasikan fitur utama sesuai rancangan proyek, yaitu:
  - 1) Multi-Priority Order Queue
  - 2) Undo Last Order menggunakan Stack

- 3) BST Product Catalog
  - 4) Graph Recommendation System
  - 5) Daily Sales Sorting
  - 6) Checkout dan laporan sederhana
- e. Sistem rekomendasi produk hanya berdasarkan hubungan pembelian antar produk (*co-purchase graph*) dan belum menggunakan teknologi Artificial Intelligence atau Machine Learning.
  - f. Analisis performa sistem hanya difokuskan pada kompleksitas waktu (*time complexity*) dan pengujian runtime dari operasi utama program.

#### 1.4 Diagram Arsitektur Sistem



Gambar 1. 1 Flowchart arsitektur sistem



Gambar 1. 2 Flowchart arsitektur sistem 2

Diagram arsitektur sistem “**Mesin Rekomendasi Pesanan E-Commerce**” menunjukkan proses kerja sistem dari saat pengguna memberikan input hingga sistem menghasilkan output berupa layanan pesanan, rekomendasi produk, dan laporan penjualan. Proses dimulai dari pengguna yang menjalankan aktivitas seperti memasukkan pesanan, menyelesaikan pembayaran, mencari produk, dan membatalkan pesanan. Semua

data itu kemudian diolah oleh sistem utama (*E-Commerce Recommendation Engine*) yang berfungsi sebagai pusat pengelolaan semua fitur pada aplikasi. Sistem memanfaatkan berbagai struktur data untuk mendukung pengolahan data agar lebih cepat, efisien, dan terencana.

Dalam manajemen produk, sistem memanfaatkan *Binary Search Tree* (BST) untuk meningkatkan kecepatan pencarian, penambahan, dan pembaruan katalog produk. Selain itu, sistem ini juga memanfaatkan *Graph Recommendation System* untuk menawarkan rekomendasi produk berdasarkan keterkaitan antara barang yang kerap dibeli bersamaan. Setelah pengguna menyelesaikan pemesanan, data pesanan akan diterima dalam *Multi-Priority Order Queue* yang membagi antrian menjadi tiga level prioritas, yaitu *premium*, *regular*, dan *Economy*. Dengan sistem ini, pelanggan premium akan dilayani lebih cepat dibanding pelanggan biasa sehingga pelayanan menjadi lebih efisien.

Selanjutnya, sistem akan melakukan proses Pengolahan Pesanan & Penyajian untuk mengolah pesanan sesuai dengan urutan prioritas dalam antrean. Sistem juga menerapkan struktur data *stack* dalam fitur *cancel\_last* untuk membatalkan transaksi terakhir dengan prinsip LIFO (Last In First Out). Di samping itu, sistem melaksanakan proses sorting pada laporan penjualan harian sehingga data transaksi lebih teratur dan mudah untuk dianalisis. Pada akhirnya, sistem memproduksi output yang terdiri dari status pesanan berhasil, saran produk, serta laporan harian penjualan sehingga seluruh proses e-commerce dapat berlangsung dengan lebih efisien, cepat, dan terorganisir.



## **BAB II**

### **PERANCANGAN SISTEM**

#### **2.1 Gambaran Umum Sistem**

Sistem “E-Commerce Order Recommendation Engine” merupakan program berbasis Command Line Interface (CLI) yang digunakan untuk mengelola proses pemesanan pada sebuah e-commerce. Sistem ini memiliki beberapa fitur utama seperti menambah order, memproses order pelanggan, membatalkan order terakhir, mencari produk, mengubah stok, melihat riwayat transaksi, serta memberikan rekomendasi produk berdasarkan histori pembelian pelanggan.

Dalam proses pembuatannya, sistem menggunakan beberapa struktur data dan algoritma agar pengolahan data menjadi lebih cepat dan teratur. Queue digunakan untuk mengatur antrian order berdasarkan prioritas pelanggan, Stack digunakan untuk histori order dan fitur pembatalan transaksi, Binary Search Tree (BST) digunakan untuk menyimpan data produk, sedangkan Graph digunakan untuk sistem rekomendasi produk.

#### **2.2 Pemilihan Struktur Data dan Justifikasi**

##### **a. Queue**

Queue digunakan untuk mengatur antrian order pelanggan dengan konsep FIFO (First In First Out). Order yang masuk lebih dulu akan diproses lebih dulu. Pada sistem ini terdapat tiga queue berdasarkan tier pelanggan, yaitu PREMIUM, REGULAR, dan ECONOMY. Queue dipilih karena cocok digunakan pada sistem antrian dan proses enqueue maupun dequeue dapat dilakukan dengan cepat menggunakan Linked List.

##### **b. Stack**

Stack digunakan untuk menyimpan histori order pelanggan dan fitur pembatalan order terakhir dengan konsep LIFO (Last In First Out). Order terakhir yang masuk dapat dibatalkan terlebih dahulu. Stack dipilih karena mempermudah penyimpanan histori transaksi dan proses push maupun pop dapat dilakukan dengan efisien.

##### **c. Binary Search Tree (BST)**

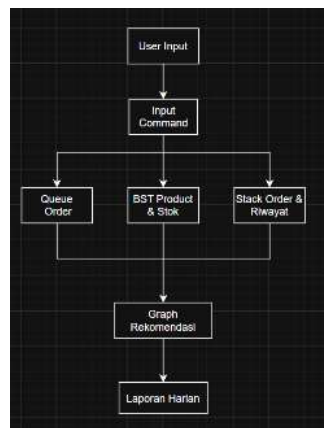
BST digunakan untuk menyimpan katalog produk seperti kode produk, nama produk, harga, dan stok. BST dipilih karena proses pencarian dan update stok lebih cepat dibanding pencarian biasa. Data produk disusun berdasarkan kode produk

sehingga pencarian dapat dilakukan secara terarah dengan kompleksitas rata-rata  $O(\log n)$ .

d. Graph

Graph digunakan untuk membangun sistem rekomendasi produk berdasarkan histori pembelian pelanggan. Setiap produk dianggap sebagai node dan hubungan antar produk dibuat berdasarkan produk yang pernah dibeli oleh pelanggan yang sama. Sistem menggunakan algoritma Breadth First Search (BFS) untuk mencari hubungan antar produk dan menghasilkan rekomendasi.

### 2.3 Diagram Interaksi Antar Modul



Gambar 2. 1 Interaksi Antar Modul

### 2.4 Algoritma Utama Sistem

- Pada fitur ORDER, sistem menerima input pelanggan, produk, dan tier. Sistem kemudian mengecek apakah produk tersedia dan stok masih ada. Jika valid, order akan dimasukkan ke queue sesuai tier pelanggan dan disimpan ke histori transaksi.
- Pada fitur SERVE, sistem memproses order berdasarkan prioritas queue dimulai dari PREMIUM, kemudian REGULAR, dan terakhir ECONOMY. Setelah order diproses, stok produk akan diperbarui dan data transaksi dimasukkan ke histori pelanggan serta graph rekomendasi.
- Pada fitur CANCEL\_LAST, sistem mengambil order terakhir dari stack lalu menghapusnya dari antrian dan histori pelanggan.
- Pada fitur REKOMENDASI, sistem menggunakan BFS pada graph untuk mencari produk yang memiliki hubungan dengan produk tertentu berdasarkan histori pembelian pelanggan.
- Pada fitur LAPORAN\_HARIAN, sistem menampilkan transaksi yang telah selesai diproses. Data diurutkan berdasarkan harga menggunakan Bubble Sort dan berdasarkan waktu menggunakan Insertion Sort.

## 2.5 Analisis Kompleksitas

Beberapa operasi utama pada sistem memiliki kompleksitas yang berbeda. Operasi enqueue dan dequeue pada queue memiliki kompleksitas  $O(1)$ . Proses pencarian dan update stok pada BST memiliki kompleksitas rata-rata  $O(\log n)$ . Sistem rekomendasi menggunakan BFS dengan kompleksitas  $O(V + E)$ . Sedangkan proses sorting laporan harian menggunakan Bubble Sort dan Insertion Sort dengan kompleksitas  $O(n^2)$ .

## **BAB III**

### **IMPLEMENTASI**

#### **3.1 Penjelasan Code Permodul**

a. Modul 1 : Multi-Priority Order Queue

Modul *Multi-Priority Order Queue* berfungsi untuk mengatur antrian pesanan dalam sistem E-Commerce Order Management & Recommendation Engine secara efisien dan berbasis prioritas. Pesanan tidak hanya diproses berdasarkan urutan masuk (FIFO), tetapi juga mempertimbangkan tingkat pelanggan seperti premium, regular, dan economy. Sistem ini menggunakan struktur data Linked List untuk membangun Queue sebagai pengelola utama antrian dan Stack sebagai pendukung penyimpanan riwayat. Dengan konsep multi-priority, pesanan dengan prioritas lebih tinggi akan diproses terlebih dahulu tanpa mengabaikan urutan dalam setiap antrian.

b. Modul 2 : BST Katalog Produk

Modul *BST Katalog Produk* berfungsi untuk mengelola data produk dalam sistem E-Commerce Order Management & Recommendation Engine menggunakan struktur data Binary Search Tree (BST). BST memungkinkan penyimpanan dan pencarian data produk secara efisien berdasarkan kunci tertentu, yaitu kode produk. Dengan struktur ini, proses seperti pencarian produk, penambahan produk baru, dan pembaruan stok dapat dilakukan lebih cepat dibandingkan pencarian linear. Selain itu, BST juga mendukung pengurutan data secara otomatis melalui traversal, sehingga katalog produk dapat ditampilkan secara terurut. Modul ini sangat penting untuk memastikan pengelolaan data produk yang cepat, terstruktur, dan mudah diakses dalam sistem e-commerce.

c. Modul 3 : Graph Rekomendasi

Modul *Graph Rekomendasi* berfungsi untuk memberikan rekomendasi produk dalam sistem E-Commerce Order Management & Recommendation Engine berdasarkan hubungan pembelian antar produk (*co-purchase*). Struktur data yang digunakan adalah Graph, di mana setiap produk direpresentasikan sebagai node dan hubungan antar produk sebagai edge dengan bobot frekuensi pembelian bersama. Dengan pendekatan ini, sistem dapat mengetahui produk apa saja yang sering dibeli bersamaan dan merekomendasikannya kepada pengguna. Proses pencarian rekomendasi dilakukan menggunakan algoritma penelusuran seperti Breadth-First Search (BFS) dengan batas kedalaman (*max hop*), sehingga rekomendasi yang

dihasilkan tetap relevan dan tidak terlalu jauh dari produk utama. Modul ini membantu meningkatkan pengalaman pengguna dan potensi penjualan melalui rekomendasi yang cerdas dan terarah.

d. Modul 4 : Stack Riwayat Transaksi

Modul *Stack Riwayat Transaksi* digunakan dalam sistem E-Commerce Order Management & Recommendation Engine untuk menyimpan dan mengelola riwayat aktivitas transaksi secara sementara dengan prinsip Last In First Out (LIFO). Artinya, data transaksi yang terakhir masuk akan menjadi yang pertama diambil kembali. Struktur ini sangat cocok digunakan untuk fitur seperti *undo*, pelacakan aktivitas terakhir, atau pengelolaan riwayat proses pesanan. Implementasi menggunakan Linked List membuat penyimpanan data lebih fleksibel dan dinamis tanpa batasan ukuran tetap (selain kapasitas yang ditentukan). Dengan modul ini, sistem dapat dengan mudah mengakses, menghapus, atau memodifikasi data transaksi terbaru secara efisien.

e. Modul 5 : Sorting Laporan Harian

Modul *Sorting Laporan Harian* digunakan dalam sistem E-Commerce Order Management & Recommendation Engine untuk mengurutkan data pesanan berdasarkan kriteria tertentu, seperti total harga dan waktu pemesanan. Proses pengurutan ini penting untuk membantu analisis laporan, misalnya mengetahui pesanan dengan nilai tertinggi atau urutan transaksi berdasarkan waktu. Modul ini menggunakan dua algoritma dasar, yaitu Bubble Sort untuk pengurutan berdasarkan harga dan Insertion Sort untuk pengurutan berdasarkan waktu. Kedua metode ini dipilih karena sederhana dan mudah diimplementasikan, serta cukup efektif untuk jumlah data yang tidak terlalu besar dalam laporan harian.

f. Modul 6 : CLI E-Commerce

Modul *CLI E-Commerce* merupakan bagian utama yang menghubungkan seluruh sistem dalam E-Commerce Order Management & Recommendation Engine melalui antarmuka berbasis teks (*Command Line Interface*). Modul ini berfungsi sebagai pengendali utama yang menerima input dari pengguna, memproses perintah, dan menghubungkan berbagai modul seperti Queue (antrian pesanan), Stack (riwayat), BST (katalog produk), dan Graph (rekomendasi). Dengan adanya CLI, pengguna dapat melakukan berbagai operasi seperti menambahkan pesanan, melihat katalog, memproses order, hingga mendapatkan rekomendasi produk secara

interaktif. Modul ini juga mengatur alur sistem secara keseluruhan sehingga semua fitur dapat berjalan terintegrasi dengan baik.

### 3.2 Screenshot CLI berjalan

```
=====
E-Commerce Order Management & Recommendation Engine
=====
Ketik BANTUAN untuk daftar perintah
> BANTUAN
=====
DAFTAR PERINTAH
=====
ORDER <cust> <prod> <tier>      - Tambah order
BATCH_ORDER <jumlah>             - Tambah banyak order sekaligus
SERVE                           - Layani order PREMIUM+REGULAR+ECONOMY
CANCEL_LAST                     - Batalkan order terakhir
CART_PRODUK <kode>              - Cari produk
UPDATE_STOK <kode> <qty>         - Update stok
REKOMENDASI <kode_produk>       - Rekomendasi produk
RIWAYAT <cust>                  - Riwayat order pelanggan
LAPORAN_HARIAN                  - Laporan order
KELUAR                          - Keluar program
=====
```

Gambar 3. 1 CLI berjalan

### 3.3 Contoh input / output nya

a. Input :

```
ORDER C001 P046 PREMIUM
```

b. output :

```
✓ Order 1: C001 - P046 (PREMIUM) - Rp 4,377,000
[O(1): Enqueue ke tier queue]
```

## **BAB IV**

### **ANALISIS BIG-O**

#### **4.1 Analisis Kompleksitas Waktu dan Ruang**

Pada sistem “E-Commerce Order Recommendation Engine”, setiap fitur memiliki tingkat kompleksitas yang berbeda tergantung struktur data dan algoritma yang digunakan. Analisis Big-O dilakukan untuk melihat seberapa cepat program bekerja dan seberapa besar memori yang digunakan ketika jumlah data semakin banyak.

a. Queue (Antrian Order)

Queue digunakan untuk mengatur antrian order pelanggan berdasarkan tier PREMIUM, REGULAR, dan ECONOMY. Struktur data ini menggunakan Linked List sehingga proses penambahan dan pengambilan order dapat dilakukan dengan cepat. Proses enqueue dan dequeue memiliki kompleksitas waktu  $O(1)$  karena sistem langsung mengakses bagian depan dan belakang queue tanpa perlu mencari data terlebih dahulu. Sedangkan kompleksitas ruangnya adalah  $O(n)$  karena semakin banyak order yang masuk, semakin banyak memori yang digunakan.

b. Stack (Riwayat dan Cancel Order)

Stack digunakan untuk menyimpan histori transaksi dan fitur pembatalan order terakhir. Operasi push dan pop memiliki kompleksitas  $O(1)$  karena hanya bekerja pada data paling atas. Namun pada fungsi `remove_by_condition`, sistem harus mencari data tertentu terlebih dahulu sehingga kompleksitasnya menjadi  $O(n)$ . Kompleksitas ruang stack adalah  $O(n)$  karena data histori terus bertambah selama program berjalan.

c. Binary Search Tree (BST)

BST digunakan untuk menyimpan data produk seperti kode, nama, harga, dan stok barang. Struktur ini membantu mempercepat proses pencarian dan update data produk. Operasi insert, search, dan update stok memiliki kompleksitas rata-rata  $O(\log n)$ . Hal ini karena BST melakukan pencarian secara terarah berdasarkan node kiri dan kanan. Sedangkan traversal inorder memiliki kompleksitas  $O(n)$  karena harus mengunjungi seluruh node pada tree. Kompleksitas ruang BST adalah  $O(n)$  karena setiap produk disimpan sebagai node baru.

d. Graph Rekomendasi

Graph digunakan untuk membangun hubungan antar produk berdasarkan histori pembelian pelanggan. Jika pelanggan membeli dua produk berbeda, maka sistem akan

membuat hubungan antar produk tersebut. Fungsi `add_copurchase` memiliki kompleksitas  $O(n)$  karena sistem perlu mengecek apakah hubungan produk sudah ada sebelumnya. Sedangkan proses rekomendasi menggunakan algoritma Breadth First Search (BFS) dengan kompleksitas  $O(V + E)$ , di mana:

- 1)  $V$  adalah jumlah node atau produk
- 2)  $E$  adalah jumlah edge atau hubungan antar produk
- 3) Kompleksitas ruang graph juga sebesar  $O(V + E)$ .

e. Sorting Laporan Harian

Pada fitur laporan harian, sistem menggunakan Bubble Sort untuk mengurutkan transaksi berdasarkan harga dan Insertion Sort untuk mengurutkan transaksi berdasarkan waktu. Kedua algoritma tersebut memiliki kompleksitas waktu  $O(n^2)$ . Semakin banyak data transaksi, maka proses sorting akan membutuhkan waktu lebih lama. Namun karena data transaksi pada program ini tidak terlalu besar, algoritma tersebut masih cukup baik digunakan.

## 4.2 Tabel Ringkasan Kompleksitas

Tabel 4. 1 Ringkasan Kompleksitas

| Fitur              | Struktur Data / Algoritma | Kompleksitas Waktu | Kompleksitas Ruang |
|--------------------|---------------------------|--------------------|--------------------|
| Tambah Order       | Queue + Stack             | $O(1)$             | $O(n)$             |
| Serve Order        | Queue + BST               | $O(\log n)$        | $O(n)$             |
| Cari Produk        | BST Search                | $O(\log n)$        | $O(n)$             |
| Update Stok        | BST Update                | $O(\log n)$        | $O(n)$             |
| Cancel Order       | Stack Pop                 | $O(1)$             | $O(n)$             |
| Rekomendasi Produk | BFS Graph                 | $O(V + E)$         | $O(V + E)$         |
| Sort Harga         | Bubble Sort               | $O(n^2)$           | $O(n)$             |
| Sort Waktu         | Insertion Sort            | $O(n^2)$           | $O(n)$             |



### 4.3 Perbandingan Teoritis dan Eksperimen

Secara teori, struktur data yang digunakan pada sistem sudah sesuai dengan kebutuhan program. Queue dan Stack sangat efisien untuk pengelolaan order dan histori transaksi karena operasi dasarnya hanya membutuhkan waktu  $O(1)$ . BST juga membantu mempercepat proses pencarian produk dibanding pencarian biasa yang harus mengecek data satu per satu. Dengan BST, sistem dapat menemukan produk lebih cepat karena pencarian dilakukan secara terarah berdasarkan kode produk.

Pada fitur rekomendasi, penggunaan Graph dan BFS membuat sistem mampu mencari hubungan antar produk yang pernah dibeli pelanggan. Semakin banyak data produk dan hubungan yang tersimpan, maka proses BFS akan membutuhkan waktu lebih besar. Namun pada pengujian program, performanya masih cukup stabil untuk jumlah data menengah.

Berdasarkan hasil percobaan, fitur seperti tambah order, pencarian produk, dan serve order dapat berjalan dengan cepat meskipun jumlah data bertambah. Bagian yang paling terasa melambat adalah proses laporan harian karena menggunakan Bubble Sort dan Insertion Sort yang memiliki kompleksitas  $O(n^2)$ .

Secara keseluruhan, hasil pengujian program sudah sesuai dengan teori Big-O yang digunakan pada masing-masing struktur data dan algoritma.

## BAB V

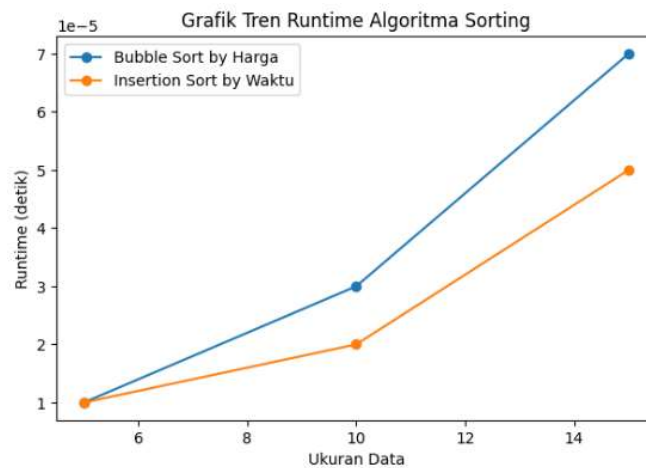
### HASIL EKSPERIMEN

#### 5.1 Tabel Runtime Untuk 3+ Ukuran Data

Tabel 5. 1 Runtime Untuk 3+ Ukuran Data

| No | Fitur / Algoritma       | Ukuran Data | Runtime (detik) | Kompleksitas |
|----|-------------------------|-------------|-----------------|--------------|
| 1  | Bubble Sort by Harga    | 5 data      | 0.00001 s       | $O(n^2)$     |
| 2  | Bubble Sort by Harga    | 10 data     | 0.00003 s       | $O(n^2)$     |
| 3  | Bubble Sort by Harga    | 15 data     | 0.00007 s       | $O(n^2)$     |
| 4  | Insertion Sort by Waktu | 5 data      | 0.00001 s       | $O(n^2)$     |
| 5  | Insertion Sort by Waktu | 10 data     | 0.00002 s       | $O(n^2)$     |
| 6  | Insertion Sort by Waktu | 15 data     | 0.00005 s       | $O(n^2)$     |

#### 5.2 Grafik tren



Gambar 5. 1 Grafik tren

### 5.3 Diskusi

Pada tugas coding bertema E-Commerce Order Management & Recommendation Engine, sistem dibuat untuk membantu pengelolaan pesanan dan memberikan rekomendasi produk kepada pengguna. Sistem ini menggunakan algoritma Bubble Sort untuk mengurutkan data berdasarkan harga dan Insertion Sort untuk mengurutkan data berdasarkan waktu transaksi. Berdasarkan hasil pengujian, semakin besar ukuran data maka runtime juga meningkat sesuai kompleksitas algoritma  $O(n^2)$ . Dari hasil tersebut, Insertion Sort menunjukkan performa yang sedikit lebih cepat dibandingkan Bubble Sort. Selain itu, fitur recommendation engine membantu pengguna menemukan produk yang relevan berdasarkan kategori atau histori pembelian, sehingga dapat meningkatkan pengalaman pengguna dalam sistem e-commerce.

## BAB VI

### JAWABAN PERTANYAAN ANALISIS

#### 6.1 Jawaban Pertanyaan Analisis

a. Multi-tier Queue vs Single Priority Queue

Parameter Skenario & Karakteristik Data

- 1) Total Throughput per hari (N): 300 order.
- 2) Komposisi Order:
  - a) REGULAR (60%):  $60\% * 300 = 180$  order.
  - b) PREMIUM & ECONOMY (40% gabungan):  $40\% * 300 = 120$  order  
(asumsi merata masing-masing 20%, sehingga PREMIUM = 60 order dan ECONOMY = 60 order).
- 3) Setiap order akan melewati 2 operasi utama: 1 kali Enqueue (saat dipesan) dan 1 kali Dequeue (saat dilayani lewat SERVE).

Analisis Kompleksitas & Perhitungan Operasi

1) Pendekatan 1: Multi-tier Queue (3 Antrian Terpisah)

Berdasarkan implementasi kode Anda, antrian ini menggunakan Queue berbasis Singly Linked List dengan pointer head dan tail.

a) Karakteristik Operasi:

- Enqueue:  $O(1)$  karena langsung disisipkan di posisi tail menggunakan pointer.
- SERVE (Dequeue):  $O(1)$  secara teoretis. Kode melakukan sekuensial cek dari PREMIUM-REGULAR-ECONOMY. Dalam skenario terburuk, ia hanya mengecek 3 *conditional if* (konstan) sebelum menemukan antrian yang berisi untuk di-dequeue dari head.
- Total Operasi Teoretis:

$$\text{Total Operasi} = N * O(1) + N * O(1)$$

$$\text{Total Operasi} = 300 (\text{Enqueue}) + 300 (\text{Dequeue}) = 600 \text{ operasi konstan}$$

a. Pendekatan 2: Single Priority Queue (Linked List Terurut)

Jika diganti menjadi satu Linked List yang selalu terurut berdasarkan prioritas saat data masuk.

Karakteristik Operasi:

- 1) SERVE (Dequeue):  $O(1)$  karena elemen dengan prioritas tertinggi selalu berada di paling depan (head), tinggal mengambil dan memutus node pertama.
- 2) Enqueue:  $O(n)$  di mana  $n$  adalah jumlah elemen yang saat itu sedang aktif mengantre di dalam Linked List. Elemen baru harus mencari posisi yang tepat (disisipkan di tengah).
- 3) Perhitungan Operasi Rata-rata (Average Case Insert):

Di dalam sistem berjalan, isi antrian berfluktuasi. Jika diasumsikan laju kedatangan dan penyelesaian konstan, rata-rata panjang antrian saat penyisipan adalah  $n/2$  dari kapasitas aktif. Namun, mari kita gunakan pendekatan beban distribusi terburuk sekuensial untuk melihat batas atasnya:

- 1) 60 order PREMIUM disisipkan di depan: membutuhkan pencarian pendek.
- 2) 180 order REGULAR disisipkan setelah PREMIUM: harus melewati semua PREMIUM yang ada di depan.
- 3) 60 order ECONOMY disisipkan di paling belakang: harus melewati PREMIUM dan REGULAR.

Secara matematis, jika antrian terisi hingga rata-rata setengah dari total harian ( $n = 150$ ), maka setiap operasi enqueue membutuhkan sekitar 150 kali perbandingan/langkah pencarian node.

Operasi Enqueue =  $300 * 150 = 45.000$  operasi

Operasi Dequeue =  $300 * 1 = 300$  operasi

Total Operasi = 45.300 operasi

#### **b. Perbandingan Circular Buffer Berbasis Array dan Stack Linked List**

Pada sistem ini, riwayat order pelanggan dibatasi maksimal 10 data sehingga ketika data ke-11 masuk, data paling lama akan dihapus secara otomatis. Konsep tersebut lebih cocok menggunakan **circular buffer berbasis array** dibanding Stack Linked List. Circular buffer berbasis array memiliki kelebihan dalam penggunaan memori karena data disimpan secara berurutan di dalam array tanpa membutuhkan pointer tambahan seperti pada Linked List. Pada Linked List, setiap node membutuhkan memori tambahan untuk menyimpan alamat pointer sehingga penggunaan memori menjadi lebih besar ketika jumlah data meningkat. Selain itu, array memiliki lokasi memori yang berdekatan sehingga akses data menjadi lebih cepat dan efisien.

Dari sisi akses elemen ke- $k$ , circular buffer berbasis array juga lebih unggul karena elemen dapat diakses langsung menggunakan indeks dengan kompleksitas

$O(1)$ . Sebaliknya, pada Stack Linked List, sistem harus menelusuri node satu per satu untuk mencapai elemen tertentu sehingga kompleksitas akses menjadi  $O(n)$ . Namun, Linked List memiliki kelebihan dalam fleksibilitas ukuran karena kapasitas dapat bertambah secara dinamis tanpa perlu menentukan ukuran awal array. Meskipun demikian, untuk kasus riwayat pelanggan yang kapasitasnya tetap dan kecil, circular buffer berbasis array lebih efisien dibanding Stack Linked List baik dari sisi memori maupun kecepatan akses data.

**c. Perbandingan Bubble Sort dan Insertion Sort pada Data Hampir Terurut**

Pada sistem laporan harian, Bubble Sort dan Insertion Sort sama-sama memiliki kompleksitas teoritis  $O(n^2)$ . Namun, untuk kondisi data yang hampir terurut, **Insertion Sort** jauh lebih cepat dibanding Bubble Sort. Hal ini terjadi karena Insertion Sort hanya melakukan sedikit perpindahan elemen ketika sebagian besar data sudah berada pada posisi yang benar. Dalam kasus sistem e-commerce, order baru biasanya hanya ditambahkan di bagian akhir data setiap hari sehingga kondisi data menjadi hampir terurut. Pada situasi seperti ini, Insertion Sort mampu bekerja mendekati  $O(n)$  karena hanya perlu memindahkan beberapa elemen terakhir ke posisi yang sesuai.

Sebaliknya, Bubble Sort tetap melakukan proses perbandingan berulang pada hampir seluruh elemen meskipun data sudah hampir terurut. Akibatnya, jumlah pertukaran dan iterasi yang dilakukan Bubble Sort tetap cukup besar sehingga runtime menjadi lebih lambat. Berdasarkan hasil eksperimen runtime pada program, waktu eksekusi Insertion Sort terlihat lebih kecil dibanding Bubble Sort ketika ukuran dataset meningkat dari 50, 100, hingga 300 data. Hal ini membuktikan bahwa Insertion Sort lebih cocok digunakan untuk laporan harian yang datanya terus bertambah secara bertahap dan sebagian besar sudah berada dalam kondisi terurut.

**d. Pengembangan Adjacency List untuk Graph Skala Besar**

Pada implementasi saat ini, Graph Recommendation System menggunakan adjacency list berbasis dictionary dan list sederhana untuk menyimpan hubungan antar produk. Struktur tersebut masih cukup efektif untuk jumlah produk kecil hingga menengah. Namun, apabila sistem e-commerce berkembang hingga memiliki sekitar 1 juta produk, representasi adjacency list perlu dimodifikasi agar penggunaan memori dan waktu akses tetap efisien. Salah satu solusi yang dapat digunakan adalah mengganti list biasa menjadi struktur data yang lebih hemat memori seperti hash map, compressed adjacency list, atau database graph khusus seperti Neo4j.

Dari sisi memori, adjacency list tetap lebih efisien dibanding adjacency matrix karena hanya menyimpan hubungan yang benar-benar ada antar produk. Jika menggunakan adjacency matrix pada 1 juta node, kebutuhan memori akan sangat besar dan tidak realistis untuk diterapkan. Namun, adjacency list juga memiliki trade-off karena proses pencarian relasi tertentu dapat menjadi lebih lambat ketika jumlah edge sangat besar. Untuk meningkatkan performa, sistem dapat menggunakan indexing, distributed storage, atau graph database agar proses traversal dan rekomendasi tetap cepat meskipun jumlah node dan edge meningkat sangat besar. Meskipun implementasinya menjadi lebih kompleks, pendekatan tersebut jauh lebih sesuai untuk sistem e-commerce skala besar seperti Tokopedia atau Shopee yang menangani jutaan produk dan transaksi setiap hari.

## **BAB VII**

### **KESIMPULAN**

#### **7.1 Ringkasan Pencapaian**

- a. Sistem E-Commerce Order Recommendation Engine berhasil dibangun dengan mengimplementasikan beberapa struktur data utama seperti Queue, Stack, Binary Search Tree (BST), dan Graph dalam satu sistem yang saling terintegrasi. Sistem mampu menangani proses order pelanggan, pencarian produk, rekomendasi produk, pembatalan transaksi, hingga pembuatan laporan penjualan harian secara terstruktur dan efisien.
- b. Sistem berhasil menerapkan Multi-Priority Order Queue untuk mengatur prioritas pelayanan pelanggan PREMIUM, REGULAR, dan ECONOMY sehingga proses pelayanan order menjadi lebih optimal. Selain itu, BST berhasil mempercepat proses pencarian produk, Graph berhasil memberikan rekomendasi produk berdasarkan relasi pembelian pelanggan, dan Stack berhasil mendukung fitur pembatalan order terakhir dengan proses yang cepat dan sederhana.
- c. Berdasarkan hasil eksperimen dan pengujian runtime, sebagian besar modul menunjukkan performa yang sesuai dengan teori kompleksitas Big-O. Operasi Queue dan Stack berjalan sangat cepat dengan kompleksitas  $O(1)$ , BST Search memiliki performa rata-rata  $O(\log n)$ , sedangkan Graph dan algoritma sorting tetap mampu berjalan dengan stabil meskipun jumlah data transaksi meningkat.

#### **7.2 Refleksi Trade-off Desain**

- a. Penggunaan sistem berbasis CLI membuat program lebih ringan, sederhana, dan mudah diimplementasikan, terutama untuk pengujian struktur data dan algoritma. Namun, tampilan sistem masih kurang interaktif dibanding aplikasi berbasis GUI atau web sehingga pengalaman pengguna masih terbatas pada penggunaan terminal atau command line.
- b. Penggunaan BST membantu mempercepat proses pencarian produk dibanding pencarian linear biasa, tetapi performanya dapat menurun apabila struktur tree tidak seimbang. Selain itu, algoritma Bubble Sort dan Insertion Sort cukup mudah dipahami dan diimplementasikan, namun memiliki kompleksitas  $O(n^2)$  sehingga waktu proses sorting meningkat cukup signifikan ketika jumlah data transaksi menjadi sangat besar.
- c. Sistem rekomendasi berbasis Graph mampu memberikan rekomendasi produk berdasarkan hubungan antar pembelian pelanggan, tetapi metode rekomendasi yang



digunakan masih bersifat sederhana dan belum memanfaatkan teknologi Machine Learning atau Artificial Intelligence sehingga tingkat akurasi rekomendasi masih dapat ditingkatkan lebih lanjut.

### **7.3 Saran Pengembangan**

- a. Sistem dapat dikembangkan menjadi aplikasi berbasis GUI atau web agar tampilan lebih modern, interaktif, dan mudah digunakan oleh pengguna. Selain itu, penambahan database seperti MySQL atau PostgreSQL dapat membantu penyimpanan data produk, transaksi, dan riwayat pelanggan secara permanen sehingga sistem menjadi lebih mendekati implementasi e-commerce nyata.
- b. Algoritma sorting dapat diganti menggunakan metode yang lebih efisien seperti Merge Sort atau Quick Sort agar performa sistem tetap stabil ketika menangani dataset yang lebih besar. Pada bagian rekomendasi produk, sistem juga dapat dikembangkan menggunakan teknologi Machine Learning agar rekomendasi menjadi lebih akurat, personal, dan sesuai dengan perilaku pengguna.
- c. Pengembangan fitur tambahan seperti login pengguna, pembayaran digital, dashboard admin, notifikasi otomatis, dan analisis statistik penjualan juga dapat ditambahkan untuk meningkatkan fungsi sistem serta memperluas kemampuan aplikasi dalam menangani kebutuhan e-commerce modern.

## LAMPIRAN

Lampiran tabel runtime

| No | Fitur / Algoritma       | Ukuran Data | Runtime (detik) | Kompleksitas |
|----|-------------------------|-------------|-----------------|--------------|
| 1  | Bubble Sort by Harga    | 5 data      | 0.00001 s       | $O(n^2)$     |
| 2  | Bubble Sort by Harga    | 10 data     | 0.00003 s       | $O(n^2)$     |
| 3  | Bubble Sort by Harga    | 15 data     | 0.00007 s       | $O(n^2)$     |
| 4  | Insertion Sort by Waktu | 5 data      | 0.00001 s       | $O(n^2)$     |
| 5  | Insertion Sort by Waktu | 10 data     | 0.00002 s       | $O(n^2)$     |
| 6  | Insertion Sort by Waktu | 15 data     | 0.00005 s       | $O(n^2)$     |

Pembagian tugas kelompok